UNIVERSIDADE FEDERAL DO PARANÁ

SAMUEL KUTZ PARANHOS

SCIENTIFIC MACHINE LEARNING METHODS FOR THE NONLINEAR BOUSSINESQ SYSTEM OF EQUATIONS

CURITIBA
2025

ABSTRACT

Partial Differential Equations (PDEs) are the essential language of scientifc computing and many predictions, studies and models are based on solving some kind of differential equation. In this work, we study the efficacy of Scientific Machine Learning methods (SciML) for the Boussinesq System of PDEs, a shallow wave model based off the Euler Equations, from classical Hydrodynamics. This study focused on understanding the relationship between the PDE parameters, its solution and the SciML method performance, implementing Physics-Informed Neural Networks (PINNs) with and without data (semi-supervised) and Fourier Neural Operators (FNOs) (purely supervised), analyzing through comparisons to the reference solutions coming from pseudospectral classical discrete methods. We already understood that PINNs without data suffer from Spectral Bias, the tendency of neural networks of first learning the low-frequency components of the solution during the optimization process, thus making small details hard to learn. On the other hand, FNOs did not show Spectral Bias, even though it is known to exibit it too. (still more to do)

Keywords: Scientific Machine Learning; Partial Differential Equations; Boussinesq System

Contents

1 INTRODUCTION **4**

1.1 Preliminaries 5

1.1.1 Multilayer Perceptrons 5

1.1.2 Physics-Informed Neural Networks 6

1.1.3 Fourier Neural Operators 7

1.1.4 Learning the Kernel in the Frequency Domain 8

1.1.5 Loss Function 9

1 INTRODUCTION

Since the start of the scientific thinking, mathematics has been of the key languages in which we can understand the world, and, Partial Differential Equations (PDEs) are one of the main dialects we use to model and to simplify nature's complexity.

These equations do not always have simple closed-form solutions (that is, when they have), showing the need of approximated numerical solutions coming from numerical methods for PDEs in order to extract predictions, compare with data, build simulations and so on. These classical methods, widely studied for all types of equations, rely on discretizing the PDE's domain into regular grids and performing time steps.

With the ascension of Machine Learning (ML) methods and the whole ecosystem of tools built around the field, a natural extesion is to build a bridge connecting numerical solution of PDEs with ML models, a field called Scientific Machine Learning (SciML).

In this work, we aim to study the application of SciML methods for approximating the solution of the Boussinesq System of PDEs, a shallow water model for a water tank with small depth, coming from the Euler Equations model of Hydrodynamics. Our goal is to implement and evaluate the performance of two famous models of SciML: the Physics-Informed Neural Networks and the Fourier Neural Operators, which together represent the two main types of methods in SciML, investigating the impact of data and the PDE model (so called Physics-Informed) presence in training, revealing their advantages and limitations for each method.

1.1 Preliminaries

Let us present, in this section, the mathematical definition for each method we will study, as well as the Boussinesq System, which is our benchmark model of choice.

1.1.1 Multilayer Perceptrons

In this work, we focus on Multilayer Perceptrons (MLPs), popularized by Rumelhart et al. (1986), which are, in essence, affine transformations parametrized by tunable weights and biases, followed by nonlinear transformations, in which an optimization process acts to fit a dataset of values from the supposed target function.

Given a labeled dataset $\{(x_i, u_i)\}_{i=1}^{N_{data}}$, the objective is to fit the function $u_\theta(x)$ by minimizing a loss function, typically the Mean Squared Error (MSE). With $||.||_2$ denoting the Euclidian norm, we have the unconstrained optimization problem:

$$\text{MSE}(\theta) = \frac{1}{N_{data}} \sum_{i=1}^{N_{data}} ||u_\theta(x_i) - u_i||_2^2 \tag{1.1}$$

where u_θ is the Multilayer Perceptron (MLP) is a function $u_\theta : \mathbb{R}^{d_{in}} \to \mathbb{R}^{d_{out}}$ defined by the composition:

$$h^{(0)} = x \in \mathbb{R}^{d_{in}}, \tag{1.2}$$

$$h^{(l)} = \sigma\left(W^{(l)} h^{(l-1)} + b^{(l)}\right) \in \mathbb{R}^N, \quad l = 1, \ldots, L-1 \tag{1.3}$$

$$u_\theta(x) = W^{(L)} h^{(L-1)} + b^{(L)} \in \mathbb{R}^{d_{out}}. \tag{1.4}$$

where $N, L \in \mathbb{N}$ denotes the number N of neurons in each of L layers, $W^{(1)} \in \mathbb{R}^{d_{in} \times N}$, $W^{(L-1)} \in \mathbb{R}^{N \times d_{out}}$ and $b^{(1)} \in \mathbb{R}^N$, $W^{(l)} \in \mathbb{R}^{N \times N}, b^{(l)} \in \mathbb{R}^N$ for $l = 2, ..., L-2$ are the weight matrix and bias vector of layer l, respectively, and $\sigma : \mathbb{R} \to \mathbb{R}$ is a nonlinear sigmoidal activation function applied element-wise, in other words, sigmoidal meaning $\sigma(t) \to 1$ as $t \to +\infty$ and $\sigma(t) \to 0$ as $t \to -\infty$ and elemnent-wise meaning it is applied to each component of a vector. We denote $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L}$ to be all of the network parameters.

A common result and of utmost importance is that, if the activation function is sigmoidal, then MLPs are universal approximators, meaning they can approximate any continuous function (CYBENKO, 1989).

In Figure 1.1, we show a common depiction of the neural networks. In this case, the input vector is the pair $(x, t) \in \mathbb{R}^2$ and output is $(\eta_\theta(x, t), u_\theta(x, t)) \in \mathbb{R}^2$.

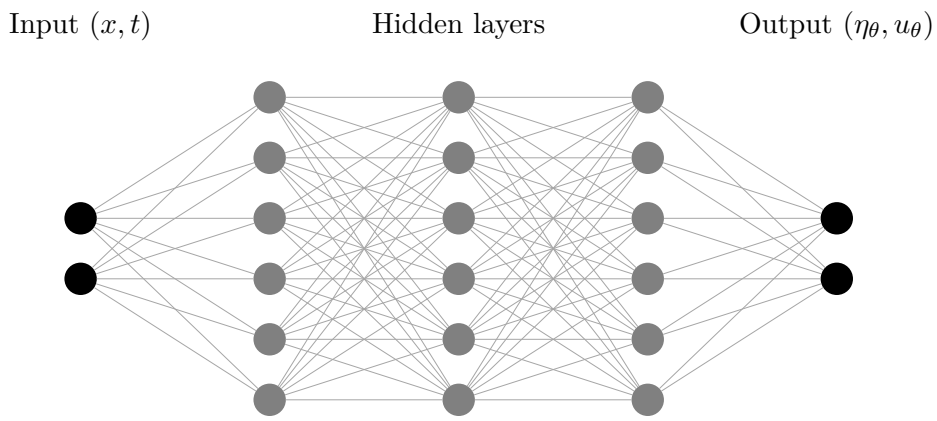

Figure 1.1 – Example of MLP architecture of input vector (x, t), with $L = 3$ hidden layers and $N = 6$ neurons on each layer.

Source: The author (2026).

1.1.2 Physics-Informed Neural Networks

Introduced by Raissi et al. (2017), Physics-Informed Neural Networks (PINNs) seeks to insert the PDE directly into the optimization process, forcing the neural network satisfy to the model equations and approximating the PDE solution.

Consider the general PDE Initial Value Problem (IVP) form as:

$$\begin{cases} \mathcal{D}[u](x,t) = 0, \quad \forall (x,t) \in \Omega \times (0,T], \\ \quad u(x,0) = u_0(x), \quad x \in \Omega \end{cases} \tag{1.5}$$

where $\mathcal{D}$ is a differential operator on u, $\Omega \times (0, T]$ is the PDE spatio-temporal domain and u_0 is the initial condition suitable for $\mathcal{D}$.

The central idea for PINNs is to incorporate (1.5) into the loss function, taking place of a data loss similar to equation 1.1.

Consider the following:

$$\mathrm{MSE}_{\mathcal{D}} = \frac{1}{N_{\mathcal{D}}} \sum_{i=1}^{N_{\mathcal{D}}} \left| \mathcal{D}[u_\theta](x_i, t_i) \right|^2, \tag{1.6}$$

where $\{(x_i, t_i)\}_{i=1}^{N_{\mathcal{D}}}$ are points in the PDEs spatio-temporal domain $\Omega \times (0, T]$.

We can also force the MLP to fit the initial condition by

$$\mathrm{MSE}_0 = \frac{1}{N_0} \sum_{i=1}^{N_0} \left| u_\theta(x_i, 0) - u_0(x_i) \right|^2, \tag{1.7}$$

where $\{x_i\}_{i=1}^{N_0}$ are points in the $\Omega \times \{0\}$ domain ($T = 0$).

In Raissi et al. (2017), the points $\{(x_i, t_i)\}_{i=1}^{N_\mathcal{D}}$ and $\{x_i\}_{i=1}^{N_0}$ are called collocation points at PDE's domain and initial condition, respectively.

Thus, the loss function for PINNs is simply:

$$\text{MSE} = \text{MSE}_\mathcal{D} + \text{MSE}_0 \tag{1.8}$$

Due to the compositional nature of MLPs, implementation of PINNs often use Automatic Differentiation (AD) (LINNAINMAA, 1970; PASZKE et al., 2019) to compute gradients. This technique takes advantage of Chain Rule derivatives to build a computational graph of derivatives until reaching basic terms with straightforward and exact derivatives.

Also, common optimizers for the PINN loss (1.8) include the Adaptive Moment Estimation (ADAM) (KINGMA et al., 2014) and the Limited-Memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) (NOCEDAL, 1980). These two algorithms were already used by Raissi et al. (2017).

1.1.3 Fourier Neural Operators

Presented by Li et al. (2020), Fourier Neural Operator (FNO) aims to learn maps between function spaces, coming from Neural Operators (NOs), a generalizaiton of NNs for approximating maps between infinite-dimensional spaces.

The target (potentially nonlinear) operator is defined by, given $D \in \mathbb{R}^{d_{in}}$ a bounded, open set, $G^\dagger : \mathcal{A}[D, \mathbb{R}^{d_a}] \to \mathcal{U}[D, \mathbb{R}^{d_u}]$, where $\mathcal{A}$ and $\mathcal{U}$ Banach spaces, a potentially infinite-dimensional complete vector space with respect to some norm (KREYSZIG, 1978),

We will study the example coming from a parametrized PDE with real parameters, approximating the operator that takes a parameter value and outputs the PDE solution to that parameter.

The Fourier Neural Operator, G_θ, is an iterative architecture mimicking a Neural Network of the form equation 1.3, meaning it composes linear operators followed by element-wise nonlinearities:

$$G_\theta = Q \circ \sigma\Big(W_L + \mathcal{K}_L\Big) \circ \cdots \circ \sigma\Big(W_1 + \mathcal{K}_1\Big) \circ P. \tag{1.9}$$

where, the lifting operator P maps the input function $a(x,t) \in \mathcal{A}[D, \mathbb{R}^{d_a}]$ to a higher-dimensional latent representation $P(a)(x,t) = v_0(x,t) \in \mathbb{R}^{d_c}$, where d_c is the dimension of the so called channels, usually way higher than the original dimension of u; the Spectral Layers, $\sigma\Big(W_l + \mathcal{K}_l\Big)$ for $l = 1, \ldots, L-1$, in which $W_l \in \mathbb{R}^{d_c \times d_c}$ for $l = 2, \ldots, L-2$ is a skip-conection meant for simple matrix multiplations, $\mathcal{K}_l$ is an Integral operator.

After the final layer, the projection operator Q maps this representation back to

$Q(v_\ell)(x,t) = u(x,t)$. Here, θ is meant to represent the many parameters of the G_θ operator, which will be defined along.

Usually, P and Q are parametrized by a encoding/decoding shallow neural networks, with the goal of simply raising the input into a higher dimension.

The core of the FNO is the stack of Spectral Layers. Each layer updates the last layer v_ℓ by linear transformations, followed by a non-linear point-wise activation σ:

$$v_{\ell+1}(x,t) = \sigma\Big((W_\ell\, v_\ell)(x,t) + (\mathcal{K}_\ell v_\ell)(x,t)\Big). \tag{1.10}$$

Here, W_ℓ is a point-wise operator that applies a linear transformation to the channel dimension d_c at each coordinate (x,t). Also, $\mathcal{K}_\ell$ is a general integral operator (linear) acting via a kernel function κ_ℓ:

$$(\mathcal{K}_\ell v_\ell)(x,t) = \int_D \kappa_\ell(x,t,y,\tau)\, v_\ell(y,\tau)\, dy\, d\tau, \tag{1.11}$$

where κ_ℓ is the learned kernel of the operator (usually a shallow neural network) and D is the domain of the problem.

The key point of FNO is to suppose that $\kappa_\ell(x,t,y,\tau) = \kappa_\ell(x-y,t-\tau)$. This allows the integral to be computed efficiently through the frequency domain, since it becomes a convolution (assuming time is periodic, which in our problem it is):

$$(\mathcal{K}_\ell v_\ell)(x,t) \overset{\text{FNO}}{=} \int_D \kappa_\ell(x-y,t-\tau)\, v_\ell(y,\tau)\, dy\, d\tau = \mathcal{F}^{-1}\Big(\mathcal{F}(\kappa_\ell)\cdot\mathcal{F}(v_\ell)\Big)(x,t). \tag{1.12}$$

An inverse Fourier Transform then returns the result to the physical domain. The Fourier transform used in (1.12) is simply the Fourier series, which is defined in the following.

1.1.4 Learning the Kernel in the Frequency Domain

To see how FNO learns the kernel, we examine how the convolution is calculated using the Fourier Series. For a periodic function $f(x)$ on $[0,2\pi]$, its series representation is:

$$f(x) = \sum_{k\in\mathbb{Z}} \hat{f}_k\, e^{ikx}, \quad \text{where} \quad \hat{f}_k = \frac{1}{2\pi}\int_0^{2\pi} f(x)e^{-ikx}dx.$$

One can check that the convolution results in the point-wise product of frequencies:

$$\begin{aligned}(\kappa * v)(x) &= \int_0^{2\pi} \kappa(x-y)v(y)dy \\ &= \sum_{k\in\mathbb{Z}} (2\pi\hat{\kappa}_k\hat{v}_k)e^{ikx}.\end{aligned}$$

This is the central idea of FNO. Instead of learning the kernel function κ_ℓ in the physical domain, FNOs directly parametrizes its Fourier coefficients, discarding learning the whole kernel as a neural net. The learnable complex-valued weights $R_{\ell,k}$ are precisely this parametrization:

$$R_{\ell,k} \approx 2\pi\hat{\kappa}_{\ell,k}.$$

In practice, computationally, this operation is performed only on a truncated set of the most important, low-frequency modes, where $|k| \leq k_{\max}$, which is a hyperparameter.

1.1.5 Loss Function

The training objective is to minimize the error between the network's prediction $G_\theta(c_i)$ and the observed solution u_i from the dataset. The loss function used is the relative L^2-norm. For functions, this is defined on the continuous domain D:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \frac{\|G_\theta(c_i) - u_i\|_{L^2(D)}}{\|u_i\|_{L^2(D)}}. \tag{1.13}$$

However, in the implementation, both $G_\theta(c_i)$ and u_i are discretized as matrices (grid 64x64) converted into vectors, much how is done in works with images, with the relative error calculated with the usual L^2-norm.

References

CYBENKO, George. Approximation by superpositions of a sigmoidal function. **Mathematics of control, signals and systems**, Springer, v. 2, n. 4, p. 303–314, 1989.

KINGMA, Diederik P.; BA, Jimmy. **Adam: A method for stochastic optimization**. 2014. Available from: <https://arxiv.org/abs/1412.6980>. Visited on: 31 Aug. 2025.

KREYSZIG, Erwin. **Introductory Functional Analysis with Applications**. New York: Wiley, 1978. ISBN 978-0471507314.

LI, Zongyi et al. Fourier neural operator for parametric partial differential equations. **arXiv preprint arXiv:2010.08895**, 2020.

LINNAINMAA, Seppo. **The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors**. 1970. Dissertação (Mestrado) – University of Helsinki, Helsinki, Finland.

NOCEDAL, Jorge. Updating quasi-Newton matrices with limited storage. **Mathematics of computation**, v. 35, n. 151, p. 773–782, 1980.

PASZKE, Adam et al. PyTorch: An imperative style, high-performance deep learning library. In: ADVANCES in Neural Information Processing Systems. [S.l.: s.n.], 2019. v. 32, p. 8026–8037.

RAISSI, Maziar; PERDIKARIS, Paris; KARNIADAKIS, George Em. **Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations**. 2017. Available from: <https://arxiv.org/abs/1711.10561>. Visited on: 31 Aug. 2025.

RUMELHART, David E; HINTON, Geoffrey E; WILLIAMS, Ronald J. Learning representations by back-propagating errors. **Nature**, Nature Publishing Group, v. 323, n. 6088, p. 533–536, 1986.